Generation	Definition
1st: Legacy Mainframe Silos (1970s-1980s)	Centralized, monolithic mainframes accessed via terminals. Rigid, expensive, batch processing via custom point-to-point connections.
2nd: Client-Server (1990s-early 2000s)	Two-tier or three-tier models splitting database engines from desktop GUIs. Needed custom middleware or EAI platforms.
3rd: Web-Based and Cloud (2010s-present)	Multi-tenant, cloud-hosted systems driven by microservices, web services (REST/SOAP) and SOA for real-time synchronization.

2. System Connectivity and Interface Options

REST (Representational State Transfer): lightweight, stateless, uses standard HTTP methods (GET, POST, PUT, DELETE), typically JSON payloads. Ideal for real-time web apps and microservices.

- Security: HTTPS/TLS and token-based authentication (OAuth 2.0, API keys). JSON has no built-in security envelope, so validation is built into the endpoint logic.

SOAP (Simple Object Access Protocol): highly structured, protocol-independent standard that relies exclusively on XML schemas (WSDL) for strict contract enforcement.

- Security: built-in WS-Security (message-level encryption, digital signatures) and WS-AtomicTransaction. Best for high-security banking and enterprise ERP transactions.

3. Data Mapping and Transformations

Connecting an ERP to an external platform (CRM, warehouse database) requires mapping fields and converting formats between schemas.

- **Data identification:** pinpointing the exact records required for a process (e.g., Customer_ID, Order_Total, SKU_Quantity).
- **Transformation rules:** converting source fields into the destination schema:
  - **Data type casting:** converting a data type (string date "2026-08-12" to epoch timestamp).
  - **Value mapping:** translating system codes (CRM "Pending_Approval" to ERP "Code_02").
  - **Aggregation and structuring:** nesting line items into a unified JSON/XML array for single-payload operations.

4. The 5 Key Planning Questions

1. **Transactions:** how to maintain ACID compliance or manage multi-system rollbacks when an operation fails midway.
2. **Connectivity:** which protocols (REST, SOAP, SFTP, AMQP) and integration styles (point-to-point, middleware/ESB, event-driven) to use.
3. **Interfaces and Data Formatting:** which data payloads and transformation rules map source to destination.
4. **Security and Compliance:** how to authenticate requests, encrypt data in transit/at rest, meet data protection standards.
5. **Error Handling and Resiliency:** how to log errors, retry failed requests, alert administrators.

5. Case Study: Standard vs. Custom Endpoint Design

Problem: standard out-of-the-box CRUD interfaces can create transactional risk, e.g., an account is created but the order submission fails, leaving orphan data.

Custom endpoint approach: one consolidated OMS endpoint that ingests account and order data in a single request, validates everything first, and rolls back automatically if any part fails.

Tradeoff: reduces network overhead and transaction risk, but increases upfront development time.

Lecture 2: Program Execution

Systems Integration: Logical vs. Physical Architectures and Implementation

1. Logical vs. Physical Systems Integration

Logical integration (human and business level): developing information systems that permit secure data sharing among stakeholders based on role authorization and operational need. Gives access to shared data across departments (Finance, Marketing, Supply Chain). Requires organizational changes, revised workflows and redefined roles to eliminate silos.

Physical integration (technical level): seamless connectivity between heterogeneous application systems across diverse platforms, OSs (Windows, UNIX) and DBMSs (Oracle, MS SQL, MySQL).

Middleware: software that presents a unified data view to end-users while maintaining data integrity and database synchronization across disparate back-end applications.

2. Business Process Re-engineering (BPR) and Organizational Dynamics

Physical integration alone cannot guarantee success; human and structural barriers are addressed through **BPR**.

- **People barrier:** interdepartmental turf issues. Management educates staff and shifts focus from departmental targets to organizational goals.
- **Cross-functional management structures:** fluid, matrix-style management where employees (e.g., budget analysts) work across functional areas, giving agility.
- **Market agility and supply chain synchronization:** connects demand-side and supply-side in real time, preventing under-stocking or over-stocking (examples: Amazon, Dell).

3. The 8 Steps in System Integration

Step	Phase	Description
1	Resource Categorization	Audit and catalog all hardware, software, OS platforms and IS architectures.
2	Compliance and Standards	Verify support for universal standards (e.g., JDBC/ODBC).
3	Legacy Systems Support	Migration and interface policies to support and encapsulate legacy assets.
4	Middleware Selection	Deploy middleware for interim and long-term communication between legacy and modern systems.
5	Authentication and Authorization	Single Sign-On (SSO) and unified access policies.
6	Centralized IT Services	Centralized IT support and help desk across scattered sites.
7	[illegible]	[illegible]

7	Backup, Recovery and Security	Disaster recovery, data redundancy and security policies.
8	Standardization Policies	Long-term procurement guidelines for future hardware/software.

4. Strategic Benefits and Limitations

Key benefits:

- **Increased revenue and growth:** lowers operational costs and inventory overhead via automated workflow synchronization.
- **Competitive parity:** lets small-to-medium enterprises join B2B supply chains via standardized electronic data exchange.
- **Enhanced information visibility:** real-time data for immediate decisions (e.g., instant credit approvals).
- **Hardware and software standardization:** long-term reduction in maintenance overhead.

Key limitations:

- **High initial capital outlay:** high upfront cost for hardware, licensing and re-engineering.
- **Power and interdepartmental conflicts:** friction from data sharing without change management.
- **Long-term / intangible ROI:** returns delayed by years, indirect benefits.
- **Creativity constraints:** strict standardization restricts local customization.

Lecture 3: Program and Project Management in Enterprise Systems

Program Management, PMO Operations, Change Control, and Critical Success Factors

1. The Role of Program Management and the PMO

Project Management Office (PMO): ensures cross-functional project teams work cohesively and address functionality issues in a timely, open and efficient manner. If teams fragment or work in silos, the entire rollout is delayed.

2A. Decision-Making Governance

A well-defined decision-making process that minimizes disputes about scope, efficiency and productivity. Teams must set early how and when decisions are finalized. Without it, trade-offs are not evaluated, causing lack of user buy-in.

2B. Project Scope and Scope Creep

Scope: the precise deliverables of a project.

Scope creep: unmanaged, continuous changes to the parameters of the original project goals. It hurts Cost, Time and Quality.

- Prevention: define parameters in a Business Case, Project Charter or Mission Statement.
- **Change Control Board (CCB):** senior management and key system users who review and authorize scope modifications.

2C. Cross-Functional Teamwork

- **Internal IT:** institutional knowledge, but may lack ERP/middleware experience.
- **External consultants:** experience from past rollouts at other organizations.
- **Vendor client specialists:** bridge technical gaps and give insight into software enhancements.

Knowledge transfer goal: consultants transfer technical knowledge to internal IT before offboarding, so staff can operate the system independently.

2D. Organizational Change Management

Integration alters processes, which creates user resistance. Change management relies on:

1. **Transparent communication:** explain the system's importance from executives down to staff.
2. **Aggressive training programs:** hands-on training before go-live to ease anxiety.
3. **Executive sponsorship:** active involvement of senior executives at kickoff and milestones.

3. The Change Control Process and Governance

When testing shows a feature does not meet requirements, scope changes are formally managed through change control. Changes are documented in a **white paper**:

1. **Issue identification:** description of the problem or new functionality.
2. **Technical and user implications:** impact on architecture, network and user workflows.
3. **Options and trade-offs:** advantages and disadvantages of each solution.
4. **Resource impact:** budget, staff hours and timeline estimates.
5. **Final recommendation:** formal proposal to the CCB for sign-off.

Lecture 4: Enterprise System Lifecycles and Continuous Process Improvement

SDLC vs. ERPLC, Implementation Strategies, and Cutover Deployment Models

1. SDLC vs. ERP Implementation Life Cycle (ERPLC)

SDLC: determine requirements to build custom software from scratch. Goal: a custom system for unique requirements.

ERPLC: modify business processes to fit prepackaged enterprise software. Goal: align processes with industry best practices.

Cost of customization: expensive, increases vendor support fees, complicates upgrades. Prioritize BPR over heavy code customization.

2. Implementation Plan Choices

Chosen based on budget, timeline and risk tolerance.

Strategy	Definition
Vanilla	Out-of-the-box core ERP functionality; adapt processes to the software. Minimizes cost, BPR and time.
Middle-of-the-Road	Moderate changes to core modules plus significant BPR. Balances customization and cost.
Comprehensive	Most expensive, lengthy, high-risk: full functionality, industry modules, extensive BPR, heavy customization.

3. Stages of the Traditional ERP Life Cycle

1. **Scope and Commitment:** business case, feasibility study, scope, team composition, top-management commitment.
2. **Analysis and Design:** Gap Analysis (differences between current workflows and system), change management plans, sandbox/prototype.
3. **Acquisition and Development:** buy production licenses, build infrastructure, configure security/roles, data migration mapping, preliminary training.
4. **Implementation (Go-Live):** convert from legacy to the new system, release to users, help desk monitoring.
5. **Operation:** long-term knowledge transfer, ongoing training, help desk handoffs, contract management, vendor patches.

4. System Conversion (Cutover) Deployment Strategies